

Text and content structure

Now that you can write the skeleton, the next step is filling the HTML body with content. The most common kind of content is text i.e. headings, paragraphs, lists, quotes, and code.

HTML has tags for many common kinds of text. Browsers ship with some default styles for all the tags. However you should pick the tag that matches what the text *means*, not the one whose default style happens to look right.

Headings and the document outline

Headings give a page its structure. There are six heading levels, from `<h1>` down to `<h6>` :

html

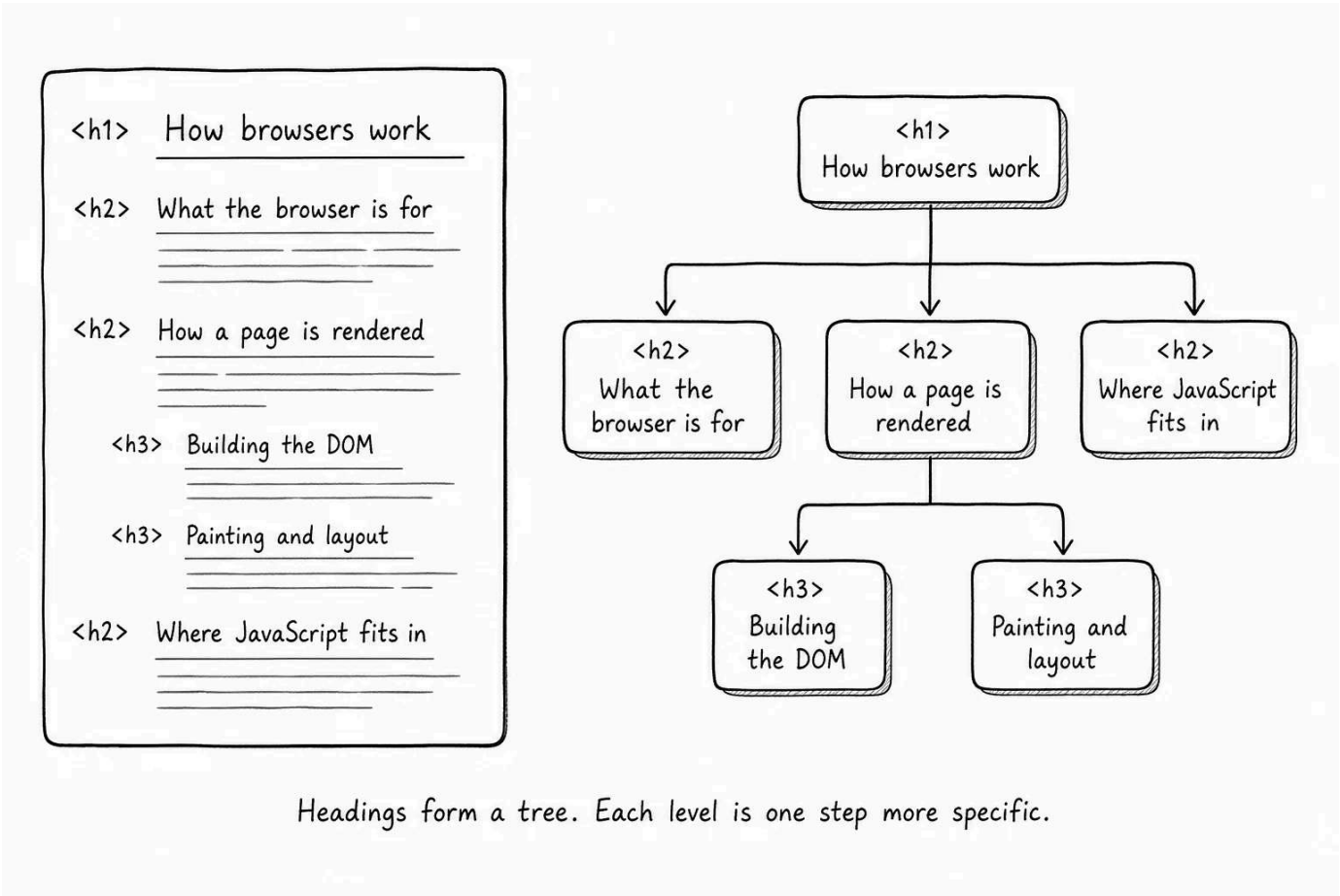
```
<h1>How browsers work</h1>
<h2>What the browser is for</h2>
<h2>How a page is rendered</h2>
<h3>Building the DOM</h3>
<h3>Painting and layout</h3>
<h2>Where JavaScript fits in</h2>
```

You can think of headings as a tree of sections. The `h1` names the whole page. Each main section is an `h2` . Inside each `h2` , you may have one or more `h3` s, and so on. The deeper you go, the more specific the topic.

A few conventions to follow:

- Use one `<h1>` per page that describes the whole page.
- Step down one level at a time. Do not jump from `<h2>` to `<h4>` .
- Use headings for real section titles, not just to make text bigger.

Hierarchy matters because screen readers, search engines, and outline tools all use it to understand the shape of the page. Skipping levels or sprinkling extra `<h2>` s to make text look bigger makes that structure harder to follow.



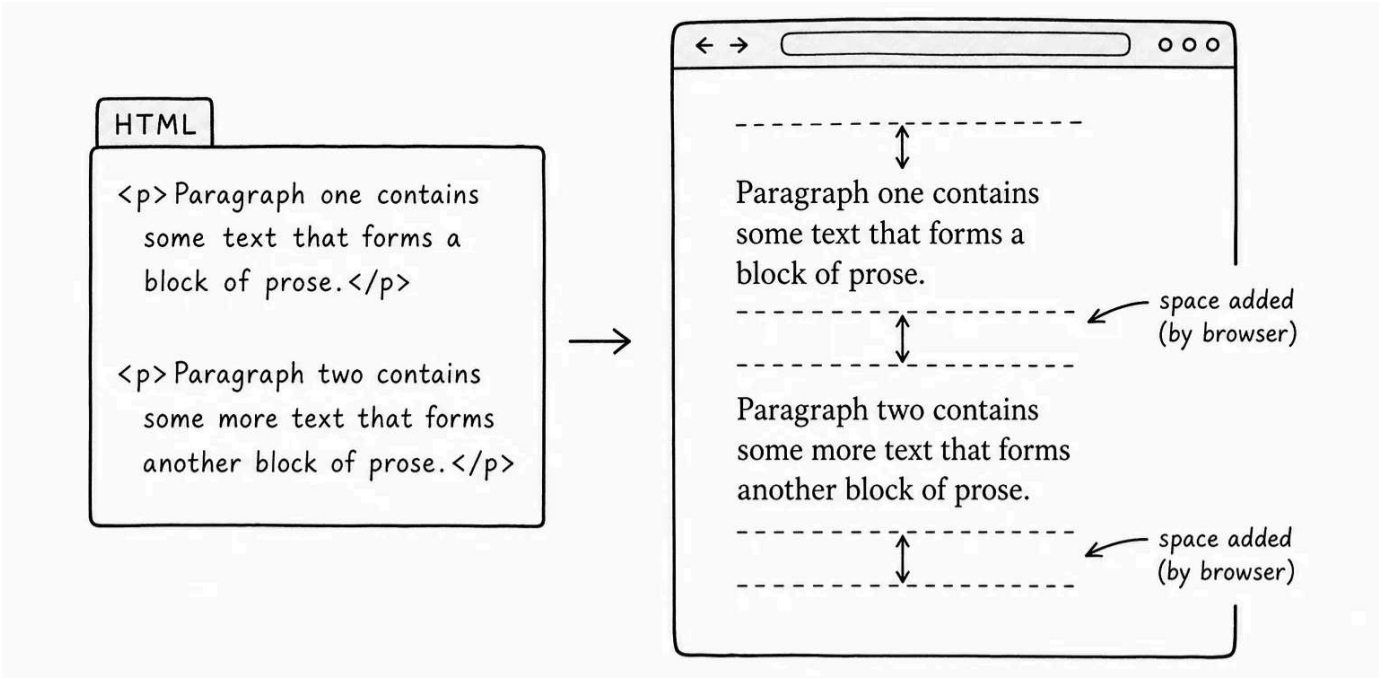
Paragraphs, breaks, and rules

A paragraph is written with the `<p>` element:

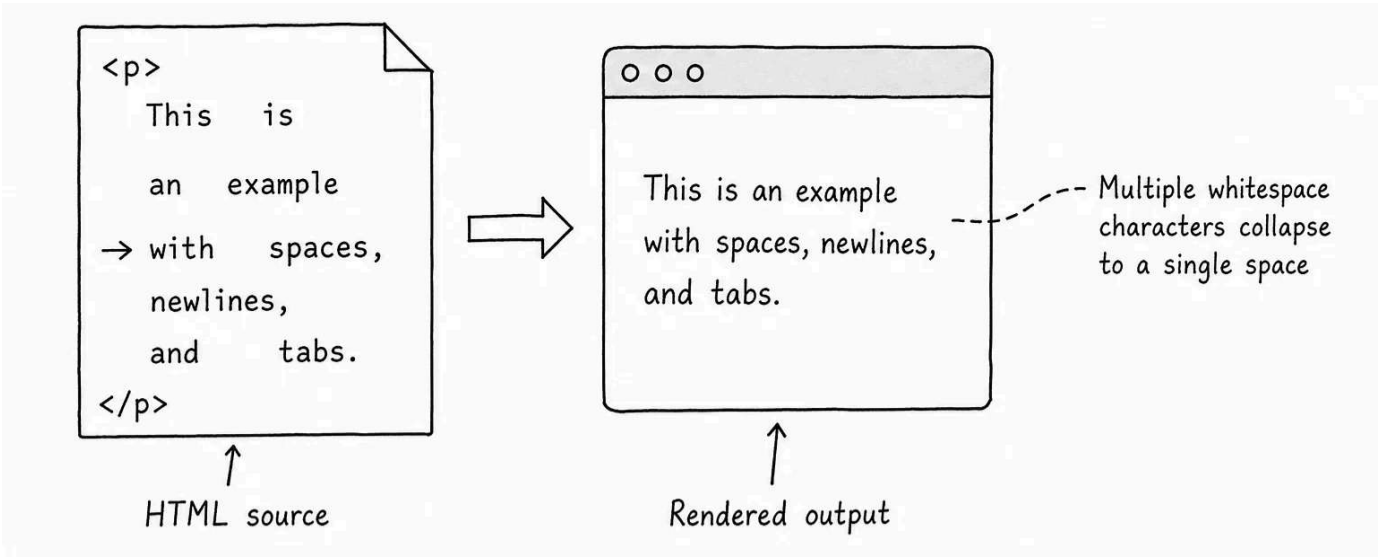
html

```
<p>HTML is a markup language. It describes the structure of a document so the browser can
render it.
<p>HTML alone is enough to put text on a page, but CSS and JavaScript usually come along
with it.
```

Paragraphs are the default container for text. The browser adds vertical space around them by default.



HTML collapses whitespace in normal text. Many spaces, tabs, and newlines in your source become a single space when rendered. That is why you do not have to format your HTML source the way you want the text to look on the page.



When you do need a line break inside a block of text, like in an address, poem, or other line-by-line text, use `<br>` :

html

```
<p>
  221B Baker Street<br>
  London<br>
  England
</p>
```

Do not use `<br>` for vertical spacing between sections. That is what paragraphs and CSS spacing are for.

The `<hr>` element draws a thematic break, usually shown as a horizontal line. Use it when you want to suggest a topic shift without adding a heading.

Emphasis and importance

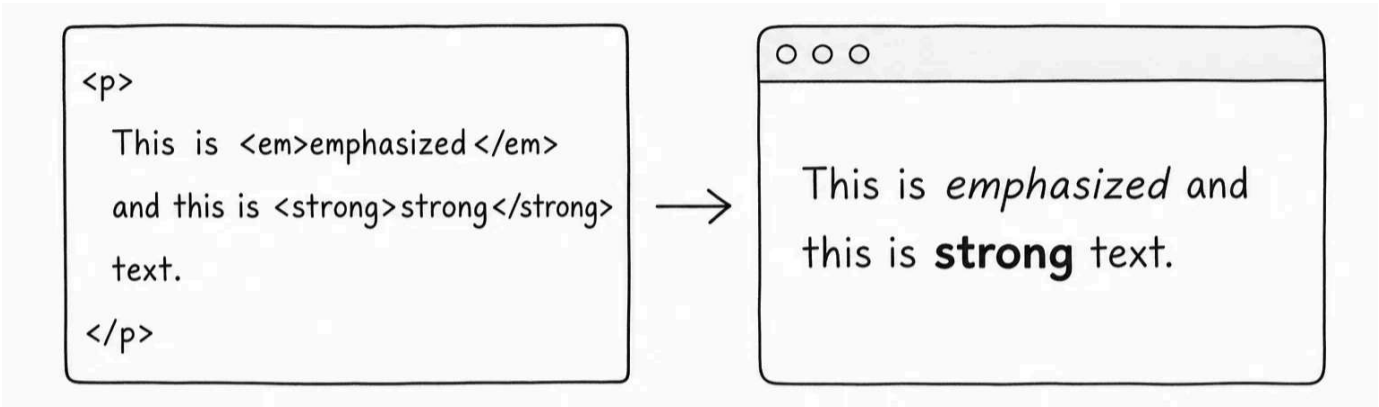
There are two elements `em` and `strong` which are used to emphasize some part of a text:

- `<em>` marks text that should be *emphasized*. Browsers render it in italics by default, but the meaning is "say this with stress when read aloud".
- `<strong>` marks text that is **important**. Browsers render it in bold by default, but the meaning is "this matters more than the surrounding text".

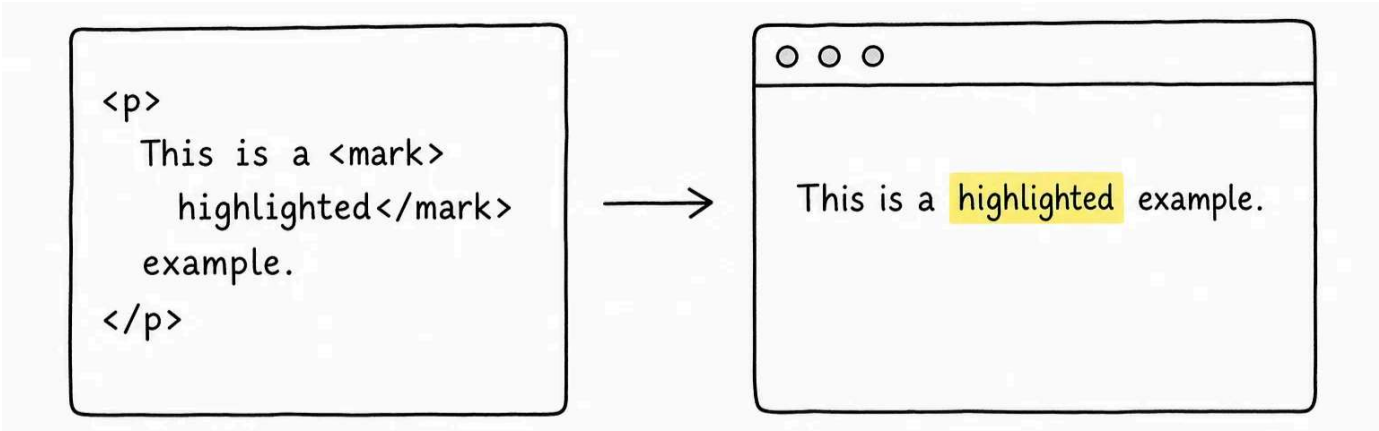
html

```
<p>You can write HTML in any text editor, but <em>save the file with a .html extension</p>
<p><strong>Never share your API keys in client-side code.</strong></p>
```

You may also see `<i>` for italic and `<b>` for bold. These still exist, but `em` and `strong` are semantically meaningful and should be preferred.



You may also use `<mark>` for text you want to highlight, like search hits. Browsers render it with a yellow background by default.



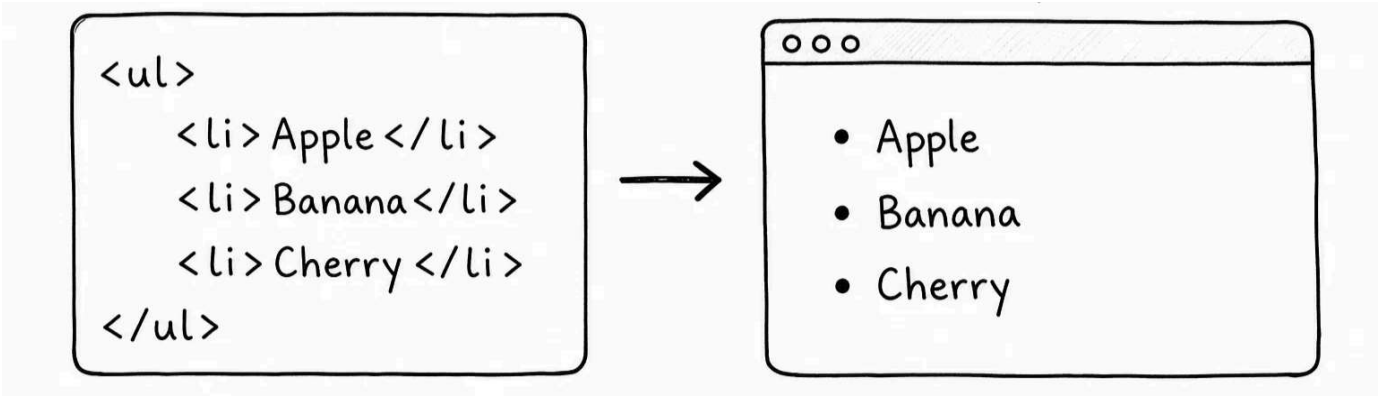
Lists

There are two main kinds of lists, plus a less common third.

`<ul>` is an unordered list. Use it when the items are a group without a meaningful order.

html

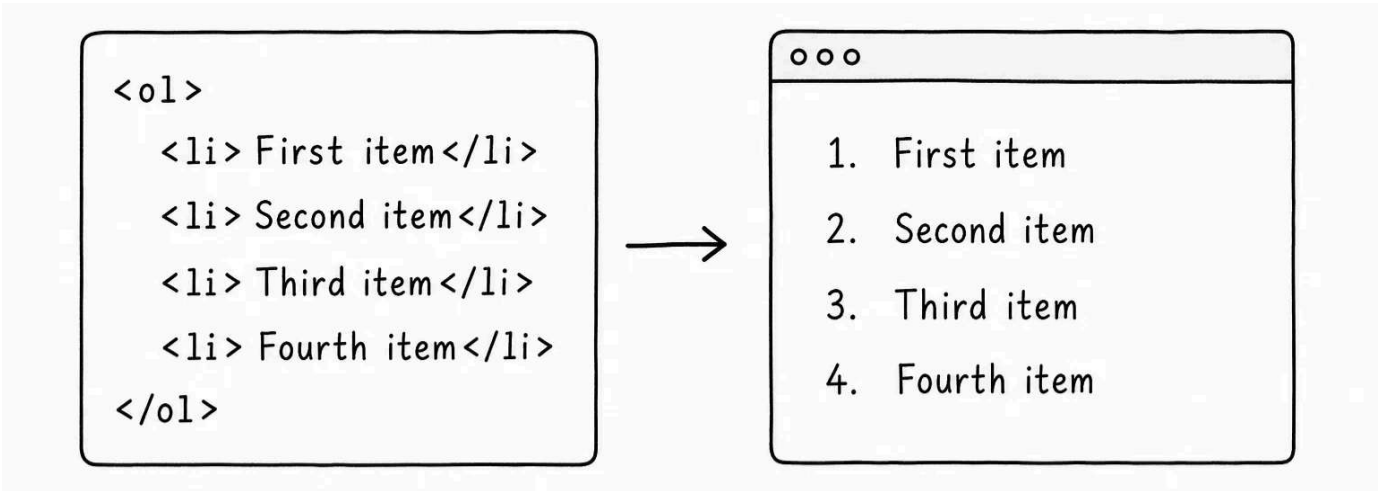
```
<ul>
  <li>Coffee</li>
  <li>Tea</li>
  <li>Water</li>
</ul>
```



`<ol>` is an ordered list. Use it when order matters, like steps in a recipe or items in a ranking.

html

```
<ol>
  <li>Open the editor.</li>
  <li>Create a file named index.html.</li>
  <li>Paste the skeleton.</li>
  <li>Open it in your browser.</li>
</ol>
```



Lists can be nested. Put a `<ul>` or `<ol>` inside an `<li>` to create sub-items:

html

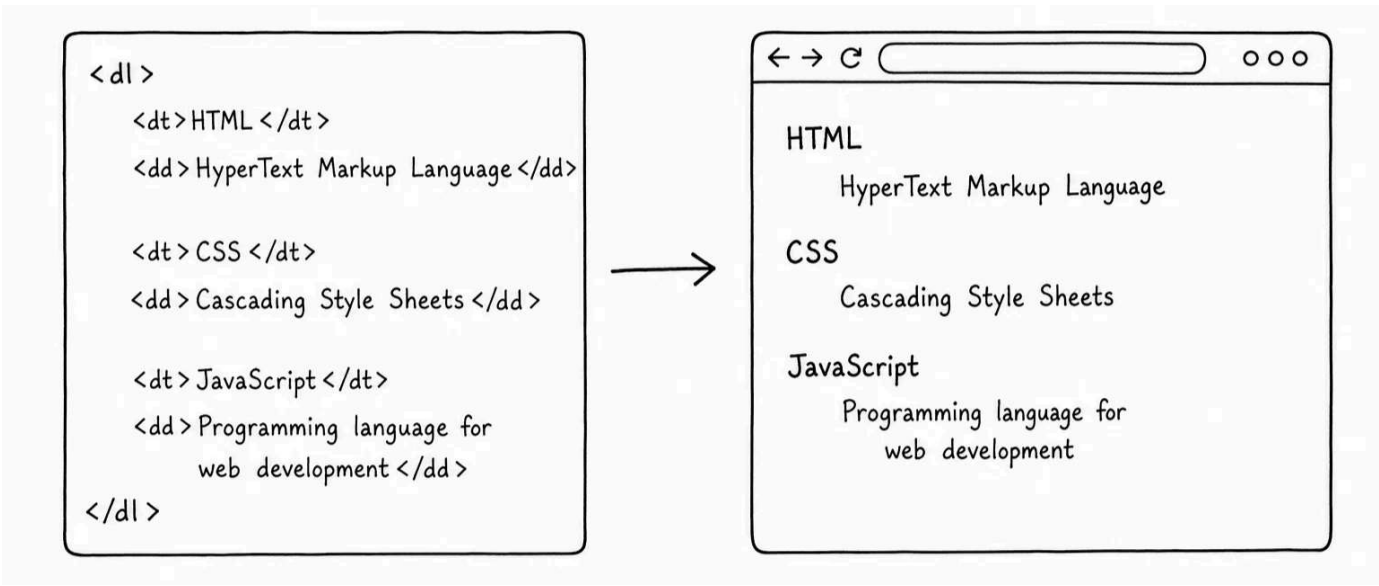
```
<ul>
  <li>Drinks
    <ul>
      <li>Coffee</li>
      <li>Tea</li>
    </ul>
  </li>
  <li>Snacks</li>
</ul>
```

There is also `<dl>` , a description list, used for name and value pairs like glossaries:

html

```
<dl>
  <dt>HTML</dt>
  <dd>A markup language for describing the structure of web pages.</dd>
  <dt>DOM</dt>
  <dd>A tree of nodes the browser builds from HTML.</dd>
</dl>
```

`<dt>` is the term, and `<dd>` is its description. You will see description lists less often than `<ul>` and `<ol>` , but they are the right choice for terms and definitions.



Quotes

Two elements mark quoted text i.e. `blockquote` and `q` .

`<blockquote>` is for longer, block-level quotes. Browsers usually indent it from the surrounding text.

html

```
<blockquote>
  <p>This paragraph is quoted from another source.</p>
</blockquote>
```

`<q>` is for short, inline quotes. Browsers add quotation marks for you, so you do not type them yourself.

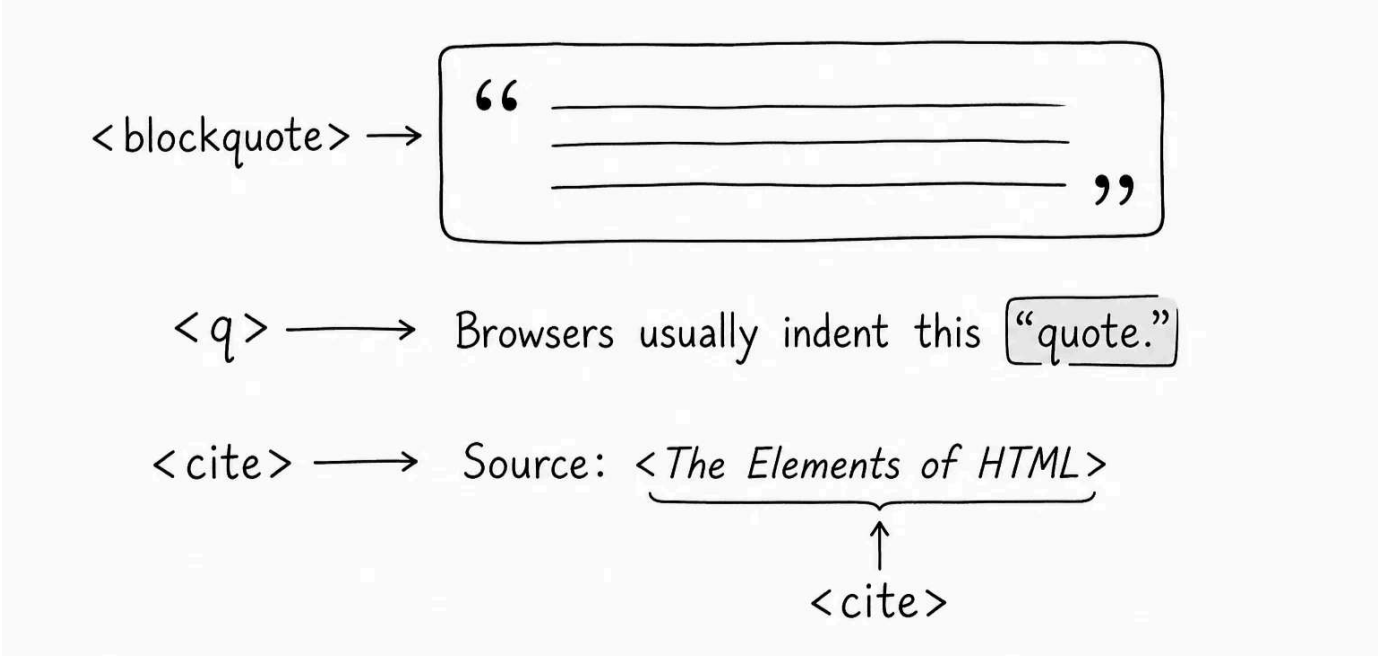
html

```
<p>She said <q>let us ship the page already</q> and closed the laptop.</p>
```

To credit the source, the `<cite>` element wraps the title of a work, such as a book, article, or paper.

html

```
<blockquote>
  <p>A clear document explains its structure before it asks the reader to remember details.
  <p><cite>A Guide to Web Documents</cite></p>
</blockquote>
```



Code in HTML

Two elements work together when you want to show code on a page.

`<code>` marks a piece of inline code. It is for short fragments inside running text. Browsers render it in a monospace font.

html

```
<p>The <code>defer</code> attribute tells the browser to keep parsing the HTML.</p>
```

`<pre>` is for preformatted blocks of text. Inside `<pre>`, the browser preserves whitespace exactly as you wrote it.

html

```
<pre>function greet(name) {
  return "Hello, " + name;
}
</pre>
```

The common combination is `<pre><code>` for code blocks:

html

```
<pre><code>function greet(name) {
  return "Hello, " + name;
}</code></pre>
```

The `<pre>` preserves indentation. The `<code>` signals that the content is source code.

Meaning over appearance

The same principle keeps coming back: pick the tag for what the text *means*, not for how it looks.

It is tempting to use `<h6>` for a small subtitle because it looks small, or `<blockquote>` to indent a paragraph because the indent is convenient. CSS lets you style anything to look any way you want. The semantic tag is what tells the browser, the screen reader, the search engine, and the next developer what the text actually is.

💡 **Stay semantic** - If you ever catch yourself picking a tag because of how it looks, swap it for a tag that matches the meaning, and use CSS for the look.

When text comes from users

If your app shows text that came from a user, two things are worth knowing now.

⚠️ **Escape user content** - Escape special characters before inserting user input into your HTML. If a user types `<script>` into a comment field and your server inserts it raw, the browser will treat it as a real script tag. Most template engines escape values by default, but it is worth confirming.

Do not reach for `<br>` every time user text contains newlines. If the text is code or terminal output, a `<pre>` block may be right. If it is normal prose,

render the escaped text in a paragraph and use CSS such as `white-space: pre-wrap` when you want newlines to display.

Try it

Create a file called `about.html` and build a short "about me" page using only the elements from this lesson:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>About me</title>
  </head>
  <body>
    <h1>Your name here</h1>
    <p>One short paragraph introducing yourself.</p>

    <h2>What I'm learning</h2>
    <ul>
      <li>HTML basics</li>
      <li>...</li>
    </ul>

    <h2>A quote I like</h2>
    <blockquote>
      <p>Put your favorite quote here.</p>
    </blockquote>

    <h2>Tools I use</h2>
    <ol>
      <li>...</li>
    </ol>
  </body>
</html>
```

Open the file in your browser and add a few real items to the lists. Wrap a word in `<em>` and another in `<strong>` inside one of the paragraphs. Open DevTools and inspect the structure. Notice that the page is readable and accessible with no CSS at all because the browser's defaults already give you a workable document.

< 0 / 6 >

Knew 0 Items

Learnt 0 Items

Skipped 0 Items

ResetProgress

Test Your Knowledge

What is the rule of thumb for picking an HTML tag?

[Click to Reveal the Answer](#)



Already Know that



Didn't Know that



Skip Question



LESSON COMPLETE

Nicely done.

Up next: Pricing Comparison Table

Next Lesson